

Finite Computations in the Dyck Symmetric Functions Appendices

1 What the Appendix Code Supports

The 2026 preprint proves a string decomposition for Dyck sequences in a range of deficits. The construction uses two local operations, called up and down, which move a Dyck sequence one step in area while preserving the relevant deficit. These operations are defined by looking for a small local pattern, removing a few entries, changing a short East or West window, and then reinserting the changed entries.

Most of the proof is symbolic: it shows that the operations preserve the Dyck condition, preserve the deficit, and stay inside the intended range. The appendix computations handle finite parts of this argument where the proof has reduced the problem to checking all possible short windows or all possible small residual inputs.

2 Appendix A

Appendix A gives reusable code for Dyck sequences. It defines the basic statistics, the skeleton tests, the insertion and extraction routines, and the local up and down maps. It also gives the routine that starts from the special skeletons and repeatedly applies up to form the lower half of each string.

The Appendix A code is included here because it is the computational form of the construction itself. It can be used to reproduce examples of the strings and to check small cases of the decomposition.

In this repository, Appendix A is represented by the following files in `code/appendix_a/`:

- `01_core_dyck_sequence_routines.py`: the basic Dyck-sequence routines and the definitions of the local up and down algorithms.
- `02_make_strings.py`: the Appendix A routine that builds the lower-half strings from the special skeletons.

3 Appendix B

Appendix B proves that the local operations used in Appendix A are well-defined in the range needed by the theorem. In the arXiv version, the four local lemmas are Lemmas 5.22–5.25:

Lemma	Title	Role
5.22	Skeleton cases succeed	Checks that the skeleton branches return Dyck sequences in the required range.
5.23	Extraction chains never fail	Checks that every extraction requested by the staged algorithms exists.
5.24	The seven-window branches do not fail	Checks the final seven-entry East and West local moves.
5.25	Bounded extraction positions and injection nonfailure	Checks the position bounds needed for later insertion steps.

These lemmas check four basic points:

- when the construction recognizes a skeleton case, the proposed output is again a Dyck sequence of the required form;
- when the construction needs to remove entries, the entries it asks for actually exist;
- when the construction reaches the seven-entry East or West move, every possible short window satisfies the inequalities needed by the proof;
- the positions used for later insertion steps stay within the allowed bounds.

The symbolic proof handles the infinite families. The finite computations in Appendix B close the remaining cases.

3.1 Residual Small Range

One checker enumerates the remaining small values $4 \leq n \leq 7$. For each Dyck sequence in that finite range, it follows the decision steps used by the up and down algorithms and confirms that each input falls into one of the cases already covered by the proof. It also confirms that the seven-entry East and West moves are not reached in this small residual range.

This corresponds to `01_residual_finite_check.py` in `code/appendix_b/`. It supports the small-range parts of Lemmas 5.22, 5.23, 5.24, and 5.25 at once:

- for Lemma 5.22, it checks the skeleton branches in the residual range;

- for Lemma 5.23, it checks that the extractions used before each branch stops exist;
- for Lemma 5.24, it confirms that the seven-window branch is not reached for $4 \leq n \leq 7$;
- for Lemma 5.25, it checks the relevant position bounds before the residual branch stops.

The successful-output transcript is `02_residual_successful_output.txt`.

3.2 Seven-Entry East and West Windows

The largest local move changes a window of seven entries. The proof reduces this part to a finite list of possible East and West windows together with the short extensions that can appear next to them. The checker enumerates those windows and verifies the threshold inequalities used in the argument. This is needed because the local move has several boundary cases that are easier and less error-prone to exhaust by computation than to list manually in the text.

This corresponds to `03_east7_west7_seven_window_checker.py` in `code/appendix_b/`. It is the finite checker for Lemma 5.24 in the $n \geq 8$ part of the proof. The successful-output transcript is `04_east7_west7_successful_output.txt`.

3.3 Position Bounds

Another part of the proof needs to know that certain extraction and insertion positions remain legal. Appendix B treats the general case symbolically and then leaves two finite domains to check. The first finite checker covers words with at most seven nonzero entries in the required range. The second covers two specific prefix forms that are excluded from the symbolic argument. In both cases, the code enumerates the finite domain and verifies that every word leads to one of the contradictions or bounds used in the written proof.

These files correspond to Lemma 5.25:

- `05_lemma_525_limited_nonzero_checker.py` checks the enlarged finite domain with $4 \leq n \leq 13$ and at most seven nonzero entries. Its successful-output transcript is `06_lemma_525_limited_nonzero_successful_output.txt`.
- `07_lemma_525_prefix_checker.py` checks the two excluded prefix forms in the finite range $9 \leq n \leq 16$. Its successful-output transcript is `08_lemma_525_prefix_successful_output.txt`.

4 Why These Checks Are Included

The computations are included because they are part of the proof, not just evidence for the theorem. Each computation corresponds to a finite exhaustive

step that appears after the main argument has reduced an infinite statement to a bounded list of cases. The appendix code records exactly what was checked, and the successful-output listings record the expected result of those checks.